

MIRA Workflow Durability Audit

Date: 2026-06-06

Author: Claude Code (CHARLIE), commissioned by Mike Harper

Framework: "Durable workflow vs fragile feature" — every major function should be a *machine with a dashboard, not a hidden code path*.

Concurrent-writer caveat: this is a shared checkout; HEAD and the working tree move under active peer sessions. One surface (#10 Self-Healer) was caught mid-edit — see its note. Where a claim depends on uncommitted state, it is labeled as such.

The 10-point durability framework

- A surface is a **durable workflow** (not a fragile feature) when:
1. **Name** — the action has a workflow name, not just a call-chain
 2. **Version** — the workflow is versioned
 3. **Run record** — every run gets a database record
 4. **Run fields** — every run records input, output, status, timestamps, error details
 5. **Step artifacts** — every step saves its intermediate output
 6. **Retry rules** — bounded retry with backoff
 7. **Idempotent** — re-running does not double-write or diverge
 8. **Golden fixtures** — known-good test cases catch regressions
 9. **Status view** — a simple status view exists in the app
 10. **Smoke test** — an automated check runs regularly and proves the path works end-to-end

Executive summary

Three surfaces are **actively, silently broken in production right now** — and the existing observability could not see it:

| ■ Live failure | Surface | Evidence |

| ---|---|---|

These three are not architecture problems. They are **the predictable consequence of having no run records, no status views, and no real smoke tests** — exactly the framework's missing criteria. A fragile feature fails silently; a durable workflow can't, because something is always watching the run record.

Scorecard

| # | Surface | Score | Verdict | Biggest gap | Feasibility | Best tool |

| ---|---|---|---|---|---|---|

| 2 | Knowledge Graph Building | **5/10** | Fragile | No durable per-run record; standalone TS worker exits silently on failure | MEDIUM | Celery |

| 4 | CMMS Sync / Work Orders | **6/10** | Fragile | No sync-run log; every tick evaporates to stdout | MEDIUM | Bun ticks + DB record |

| 5 | Tag Ingestion | **4/10** | Fragile | No batch record; non-idempotent; absent from CI; mostly on unmerged branch | EASY-MEDIUM | asyncio task in relay |

| 8 | Adversarial Review / QA | **5/10** | Fragile | Scheduled bot QA silently no-ops; no queryable run history | EASY-MEDIUM | GitHub Actions + NeonDB |

| 9 | Lead Discovery / HubSpot | **6/10** | Fragile | HubSpot delivery broken 1,096 runs; no status view, no canary | MEDIUM | launchd + DB + canary |

| 10 | Self-Healer / VPS Health | **4/10** | Fragile | Cannot recreate a removed container; zero tests; no dashboard | EASY | cron + fill the gap |

Cross-cutting patterns (the systemic gaps)

Tallying all 100 cells (10 surfaces × 10 criteria) reveals where MIRA is *structurally* weak — these are the gaps to fix once, platform-wide, instead of ten times:

| Criterion | YES | PARTIAL | NO | Read |

|---|---|---|---|

| 1. Name | 6 | 4 | 0 | ■ Strongest. Surfaces are nameable. |

| 2. **Version** | 0 | 2 | 8 | ■ **Universal gap. Not a single workflow is versioned.** |

| 3. **Run record** | 2 | 4 | 4 | ■ The core durable-workflow primitive is mostly absent. |

| 4. Run fields | 1 | 9 | 0 | ■ Data is captured — but **in-memory / stdout, not durably.** |

| 5. Step artifacts | 1 | 6 | 3 | ■ Intermediate state is garbage-collected after the call returns. |

| 6. Retry | 3 | 5 | 2 | ■ Best-covered "hard" criterion — provider cascades + outbox drains. |

| 8. Golden fixtures | 2 | 5 | 3 | ■ Unit tests exist; true input→output golden fixtures rare. |

| 9. **Status view** | 2 | 4 | 4 | ■ And **several "status views" are hardcoded mock data** (see below). |

| 10. **Smoke test** | 0 | 5 | 5 | ■ **Not one surface has a real, regularly-running end-to-end smoke.** |

The five recurring anti-patterns

The one primitive that fixes most of this

```
workflow_runs(  
  run_id uuid pk, workflow_name text, workflow_version text,  
  tenant_id text, status text, -- running|ok|degraded|failed  
  input jsonb, output jsonb, error_detail text,  
  started_at timestamptz, finished_at timestamptz  
)
```

Per-surface findings

1. PDF/Manual Ingest Pipeline — 5/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

|---|---|---|---|

| 5 | Step artifacts | NO | Extracted text & intermediate chunks never persisted; goes straight to OW |

| 7 | Idempotent | PARTIAL | Content-hash dedup + unique index for cloud picks; **local re-uploads create new rows** |

| 8 | Golden fixtures | NO | Unit tests use 1-byte stub PDFs; no real-PDF end-to-end golden |

Feasibility: MEDIUM (run record already exists — gaps are additive).

Tool: Trigger.dev — replaces the fire-and-forget closure with durable runs + step artifacts + retries natively (note the VPS-OOM caveat on concurrent docling).

2. Knowledge Graph Building — 5/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

|---|---|---|---|

| 5 | Step artifacts | PARTIAL | Final entities/proposals/evidence durable; raw extraction output never stored |

| 8 | Golden fixtures | PARTIAL | Strong unit tests for type/direction; no "this PDF → these entities" golden |

Biggest gap: No durable per-run record. If a nightly run silently writes zero proposals (upstream query changed), there's no alert, no history, no "when did tenant X last get KG proposals?"

Feasibility: MEDIUM (lifecycle machinery already solid — pure instrumentation).

3. Diagnostic Engine (Ask MIRA) — 6/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

---|---|---|---

Feasibility: MEDIUM (schema fully exists — 3–5 precise wiring changes + one Hub page).

Tool: None needed — async fire-and-forget trace write is architecturally correct given "never block the reply." It just needs the missing wiring.

4. CMMS Sync / Work Order Pipeline — 6/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

---|---|---|---

| 6 | Retry | YES | 3-attempt backoff on WO create; 5-min outbox drain w/ 3h alert; quota circuit-breaker 5min→1h |

| 8 | Golden fixtures | PARTIAL | Type-shape + retry tests; no full round-trip golden (push→watermark→pull→conflict) |

Biggest gap: No durable per-run record. Every sync tick evaporates to stdout; circuit-breaker state is lost on restart; the only alerting (3h outbox) covers bot-created WOs, not the main NeonDB↔Atlas loop.

Tool: Keep Bun interval-tick (correct for 60s poll) + add the DB run record. Trigger.dev only if connectors graduate to Hub-scheduled syncs.

5. Tag Ingestion (Current State) — 4/10 — Fragile feature

| # | Criterion | Verdict | Evidence (branch) |

---|---|---|---

| 4 | Run fields | PARTIAL | Handler logs tenant/source/accepted/rejected; rejected reasons only in HTTP body; no request-id |

| 6 | Retry | PARTIAL | Collector has bounded backoff; relay only guarantees partial-write safety, **not dedup on retry** |

| 8 | Golden fixtures | PARTIAL | 54 inline unit tests across ingest/collector/diff/flaky; not eval-framework golden fixtures |

Feasibility: EASY (CI, idempotency) → MEDIUM (run record, runtime wiring).

6. Connector Framework — 4.5/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

---|---|---|---

| 8 | Golden fixtures | PARTIAL | 5 realistic JSON fixtures + count assertions; no locked snapshot/diff-on-failure |

Biggest gap: No durable run record (computed → logged → GC'd) **and** the import side isn't idempotent — re-running multiplies the admin review queue (a correctness bug, not just ops).

Feasibility: MEDIUM (persistence layer already wired; ~2 days/gap).

Tool: Celery when scheduled runs are needed (already in stack via mira-ops Flower); Trigger.dev contraindicated on the 8GB VPS.

7. Document Upload → Retrieval — 4/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

---|---|---|---

| 6 | Retry | PARTIAL | Manual retry (cloud, failed-only); no auto-retry/backoff/DLQ |

| 7 | Idempotent | PARTIAL | Cloud picks deduped; **local re-uploads create new row + new OW file** |

| 9 | Status view | PARTIAL | Shows "Document indexed successfully" — **but "indexed" = OW collection, not the citable store; misleading** |

| 10 | Smoke test | NO | CI smoke covers page/auth only; no upload→retrieve round-trip |

Feasibility: HARD (new extraction runtime + chunking pass + migration + retrieval blast radius; 2–3 days clean).

Tool: Celery (already deployed) or **Trigger.dev** — current pipeline is a fire-and-forget Promise with no restart durability.

8. Adversarial Review / QA — 5/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

|---|---|---|---|

| 1 | Name | YES | Named workflows: Automated Code Review, MIRA QA Regression, Benchmark Weekly, Evaluations |

| 5 | Step artifacts | PARTIAL | QA/bench upload GH artifacts (30–90d); **code-review output is transient PR comment only** |

| 7 | Idempotent | PARTIAL | Issue-filing dedups (append comment); scorecard writer emits duplicate files per SHA |

Biggest gap: The most operationally important layer — the 2-hourly end-to-end bot QA that the README says caught every real production outage — **has never executed in CI**. The schedule is theater; the baseline it would compare against was never seeded. False-green checks, zero regression signal.

Feasibility: EASY (secret) → MEDIUM (DB persistence) → HARD (full dashboard).

Tool: **GitHub Actions** — already has schedule/artifacts/dedup. The missing piece is the persistence layer (NeonDB), not the orchestrator.

9. Lead Discovery / HubSpot Pipeline — 6/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

|---|---|---|---|

| 2 | Version | NO | No version field anywhere in lead-hunter |

| 8 | Golden fixtures | PARTIAL | 20 snippet-parser fixtures (tied to 2026-04-22 incident); no full-run golden |

| 10 | Smoke test | NO | No CI/cron canary; tests cover harness logic, not live run |

Feasibility: EASY (token, version) → MEDIUM (status view, canary).

Tool: **Keep launchd + Celery** — singleton lock + hard timeout + RunReport + per-step retry already give it Prefect-like bones. Prefect Cloud free tier would add a dashboard but is over-engineered for 1 hourly job; DB run table + Hub page + canary closes it cleanly.

10. Self-Healer / VPS Health — 4/10 — Fragile feature

| # | Criterion | Verdict | Evidence |

|---|---|---|---|

| 10 | Smoke test | PARTIAL | Heartbeat every 15min is a scheduled check; but no CI test runs the healer to verify it heals |

Feasibility: EASY (the code fix is essentially written — it needs committing, a golden test, and a dashboard).

Tool: **Keep cron + subprocess** — the healer is a bounded remediation script, not a workflow; the fix is filling the implementation gap, not adding an orchestrator.

Orchestration tool recommendation

| Tool | Where it fits in MIRA | Verdict |

|---|---|---|

| **Temporal** | Mission-critical multi-step durability (CMMS bi-directional sync, tag ingest if it grows to multi-gateway fanout). | **Not yet.** Real durability guarantees, but operationally heavy (server + workers). No surface today has the cross-process, long-running, must-not-lose-state profile that justifies it. Revisit when CMMS sync becomes revenue-critical. |

| **DBOS** | Postgres-backed durable workflows — *uniquely well-matched to MIRA's "NeonDB is the source of truth" architecture*. Run record + workflow state live in the same Postgres you already query. | **The dark-horse pick.** For any surface where the durable-run-record gap is the whole problem (KG building, CMMS sync, connectors), DBOS gives durability + idempotency + run records natively in NeonDB with no new broker/server. Strongly worth a spike — it directly addresses this audit's #1

systemic gap. |

Recommended sequence:

2. **Keep Celery** for Python background work; fold the standalone TS KG worker into it.
 3. **Re-evaluate Trigger.dev** for Hub ingest *after* mem-limits are proven (OOM gate).
 4. **Defer Temporal/Prefect** until a surface's scale or revenue-criticality actually demands them.
-

Prioritized hardening roadmap

Ordered by (**live damage × ease of fix**), not by score.

P0 — Broken in production right now (fix this week)

P1 — Silent correctness bugs (fix this sprint)

P2 — The platform primitive (the structural fix)

P3 — Observability polish

Closing assessment

MIRA is a **capable product built on fragile plumbing**. The domain logic is genuinely good — the UNS gate, groundedness scoring, KG proposal lifecycle, provider cascade, connector contracts, retry/idempotency on the write side. But almost none of it is *operationally observable*. The recurring story across all ten surfaces is identical: **the workflow runs, does real work, and then forgets it ever happened** — leaving a stdout line, an overwritten temp file, or a last-write-wins row where a durable run record should be. The three live production failures are not bad luck; they are the deterministic output of a system with no run records, no real smoke tests, and status views made of mock data.